

1) Unsupervised learning is when our data is not explicitly labeled, and we ask the algorithm to learn something meaningful on its own without labels.

2) The 2 types of unsupervised learning we will explore today are: K-means and PCA (Principal Component Analysis)

3) What is a centroid? Take some time on this question after reading under the K-Means block in the ipynb. Draw some of your understanding out on the board! Include some formulas if you can.

A centroid is a center or average position of many data points. This is shown in the Data and Clusters graph with 3 stars marking the central points around, and is a mean/average point

4) Under line 7, the blue points represent data points and the stars represent centroids.

5) Write the formula for finding the closest centroids on the board, and then summarize it using your own words here.

The centroid formula is used to assign each training sample to the closest centroid, shortest distance between the centroid and the point.

6) Read the code in line 8. Name the type of python object each of these are:

Centroids (float), X (array), np.square(method), min_distance (integer), idx (numpy array)

7) What is the purpose of line 9?

Line 9 is checking whether the function works, because line 9 returned 1, 3, 2 as the professor expected we know it is working properly

8) What is the difference between the first plot under line 7 versus the one in line 10?

The plot in line 10 assigns the data based on which centroid the point is closest to

9) On the board, write the equation under 'Computing Centroid Means' and label each of the variables. On your canvas submission, write 2 sentences on what this equation represents.

Find a cluster (various clusters) and name a cluster that grabs points that are close to each other. It then places them into a weighted average formula and weighs it by the point density/distance. Smaller for c values that are larger, larger for c values that are smaller.

10) Challenge: What is the purpose of line 17-24?

Lines 17-24 are trying to move the centroids around to find the best position for them to minimize the distance from the centroid to the points

11) Line 25 onwards shows you how to do exactly what the above lines did with hard code. That means that scikit learn has an inbuilt function for all the things we just hand-coded! Yay! Write the code that helps you do what we did faster here:

```
from sklearn.cluster import KMeans
```

```
km1= KMeans(3)
```

```
km1.fit (X1)
```

12) Look up the parameters for Kmeans and explain them here!

K - Number of clusters, telling the algorithm how many groups to form

Data Points - input observations

Centroids - Center point for each cluster

Distance Metric - Determines how distance is measured, Euclidean, Polar, etc.

13) Check out the image compression example from lines 27-36, and write at least 3 questions on the board that you have about it (if you don't have any questions, write 3 'takeaways'.

14) It's PCA time. What type of object is X3 in line 39?

Two dimensional array

15) What type of normalization are they doing for this data?

Computing the mean, subtracting that mean from the original data, and computing the standard deviation and dividing the new data by the standard deviation. Z-score formula standard normalization

16) Vectors are kinda hard. Find a resource online to help you understand #3-- I'm here to help! Insert the resource here.

<https://www.geeksforgeeks.org/data-science/singular-value-decomposition-svd/>

17) In line 42, what is the purpose of this function, and what type of objects is the function returning?

Performs Principle Component Analysis (PCA), returns a tuple of numpy arrays (mean, std, X_norm, U, S, V)

18) Don't worry about all the particularities of line 47, data visualization is it's own class and specialty, but check out the graph. Read the paragraph just below and explain it in a sentence in normal-people speak here. I would ignore the paragraph after that unless you're feeling brave.

Most of the data follows one main direction (a diagonal line), so instead of keeping both dimensions, we just keep that one important direction. We do this by projecting each point onto that line, which turns every point into a single number that tells us where it lies along it. If we want to go back to the original 2D form, we can stretch those numbers back into points, but it won't be exact because we threw away the less important direction.

19) Skip ahead to the graph under line 54. Write a sentence about what this is doing for me.

This graph shows the new 1-dimensional position placement of the points by keeping the principle direction, appearing as a single eigenvector

20) Lines 55 do the thing that happened in the above lines for PCA with an in-built function in scikit learn. Insert the code here:

```
from sklearn.decomposition import PCA  
  
pca= PCA (n_components= 2)  
  
pca_result= pca.fit_transform(X3)
```

21) In the code you just wrote above, where did X3 even come from? What is it? And what is n_components anyway?

X3 comes from ln39 is what we defined to represent the X values from our imported data. N_components is how many dimensions we want to keep after reducing the data. In this case it will keep the 2 most important dimensions

22) Somewhere before the graph are a few sentences that I would like you to comprehend:

```
pca_result: (50, 2)  
Variation per principal component: [0.87062385 0.12937615]
```

The above ratio is how much of the variance or variability in our data is captured by the two principal components.

You may have to look this up, but what does the (50, 2) mean? And in normal people speak, what does the output for Variation per principal component mean?

This means that our data transformed to 50 rows and 2 columns (2 being the n_components we defined earlier) instead of however many features we started with

Variation per principle component is how much of the important information each component keeps, basically 87% is captured by the first dimension and only 13% is captured by the second direction

23) Skip ahead to line 89, and read through the code, it's kinda cool!

24) Here's the fun now... Import the important libraries you need in a fresh ipynb, and run this code:

```
from sklearn.datasets import load_sample_image
import matplotlib.pyplot as plt

china = load_sample_image("china.jpg")
plt.imshow(china)
plt.axis('off')
```

Can you try to compress this image for me, using K-Means?

25) Extra super bonus if you are already done: Find a way to upload your own (class appropriate) image into an ipynb, and see if you can reduce the number of colors in it.